

C to Assembly and the Compilation Process

What the compiler has actually been doing to your code. Prologs, epilogs, activation records, and why a `while` loop is laid out backwards

Textbook: Dean, *Embedded Systems Fundamentals*, 2nd ed., Chapter 5, printed pages 133–165.

Lab manual: Lab 10 (stack and heap on the STM32F303), Section 10.2.

1 What we are actually after this week

Midterm is behind you. This chapter is a change of pace: nothing to configure, no registers to get wrong, just a look under the hood at what your C has been compiling into for the last nine weeks.

The question is: **what does the compiler actually produce, and why should you care?**

The "why care" has three concrete answers. You will read disassembly in a debugger when something makes no sense. You will need to know where variables physically live when your 40 KB of SRAM runs out. And you will occasionally need to make code faster or smaller, which requires knowing what costs what.

The plan:

1. **The toolchain.** Compiler, assembler, linker, programmer, debugger. Five tools, what each consumes and produces.
2. **Memory types and the memory map of a program.** What goes in flash, what goes in RAM, and how an initialised variable gets initialised.
3. **The AAPCS.** Which registers a function may clobber and which it must preserve. This one rule explains a lot.
4. **Prolog, body, epilog.** The three parts of every function, and the **activation record** on the stack.
5. **Exception handlers**, and how they differ from ordinary functions.
6. **Control flow.** `if/else`, `switch`, and the three loops, each with its actual assembly.
7. **Data access.** Static, automatic, pointer, and array, including the arithmetic for a 2D array.

Item 3 is the key to everything else and it is where the exam questions cluster. Item 6 produces the classic "explain what each instruction does" exercise, which the book's Chapter 5 exercises are made of almost entirely. Item 7 is where Lab 10 lives.

2 Deep dive 1: The toolchain

Why does a high-level language exist at all? Because writing directly in assembly means making a great many small decisions for every line: where to put each variable, which register to use, which conditional branch after a comparison, how to pass data between functions. Every one of those is an opportunity to be wrong, and none of them is the problem you were actually trying to solve.

So we hand them to a tool. Five tools, in fact.

Tool	In → out	Job
Compiler	.c → .s	C to assembly for one file at a time
Assembler	.s → .o	Assembly to object code
Linker/loader	.o + libs → .axf	Combine, place in memory, resolve addresses
Programmer	.axf → flash	Write the image into the MCU's non-volatile memory
Debugger	—	Control execution, inspect registers and memory

The compiler's own steps, in order: preprocess (paste in `#include` files, substitute `#define` macros), check syntax, build intermediate code, optimise at a high level, generate assembly and assign variables to registers, optimise at a low level, write out the .s file.

Compilation is not one-to-one

The compiler's only obligation is to produce object code that *correctly implements* what the source specifies. There are **many** correct compilations of one C program, differing in instruction choice, register allocation, memory used, and code layout.

This is worth internalising because it kills a whole class of wrong intuition. You cannot reason about your program's timing by counting C statements. Two lines that look equally cheap may differ by a factor of ten, and the same line may compile differently in two places in the same file.

Beware the trap

This is also the deep reason `volatile` exists, and it ties directly back to Week 08. The compiler is licensed to do anything that preserves the *observable behaviour of the C program*. From its point of view, an ISR modifying a variable is not part of the C program, so caching that variable in a register preserves behaviour perfectly. It is only wrong from your point of view. `volatile` is how you widen the compiler's definition of observable.

The assembler consumes a file made of instructions plus a few other elements:

- An **instruction**: a mnemonic and its operands. `BLX R0` encodes to the 16-bit value `0x4780`.
- A **comment**, starting with a semicolon, ignored entirely.
- A **symbol**: a text name representing a value, like `Stack_Size`.
- A **label**: a symbol representing an *address*, the current location in memory. `Reset_Handler` and `__initial_sp` are labels.

Object code with unresolved **external references** is incomplete; the assembler lists them and the linker resolves them once it knows where everything landed.

In the lab

Your labs use VS Code with CMake and the GCC ARM toolchain rather than the book's Keil `armcc` and `armlink`, so the tool names differ: `arm-none-eabi-gcc`, `arm-none-eabi-ld`, and `arm-none-eabi-objdump` for disassembly. The stages are identical.

Two practical commands worth knowing. `arm-none-eabi-objdump -d your.elf dis-`

assembles, which is how you see the listings in this chapter for your own code. And `arm-none-eabi-size your.elf` prints your text, data, and bss sizes, which is how you find out whether you are about to run out of the 40 KB of SRAM that Lab 10 makes you think about.

3 Deep dive 2: Where things live

3.1 The memory types

Type	Volatile?	Notes
ROM (mask)	No	Contents fixed at fabrication
EEPROM	No	Erase and reprogram one location at a time. Slow
Flash EEPROM	No	Erase and program a whole page at a time. Faster
SRAM	Yes	Built from latches. Fast, holds data until power is lost
DRAM	Yes	A transistor acting as a capacitor. Slower, needs periodic refresh, but the cell is much smaller so it is much cheaper

MCUs typically integrate flash ROM and SRAM. Some add EEPROM for configuration data that must persist but also change. DRAM is rare on-chip because the refresh controller costs more than the density saves at these sizes.

Book board vs your board

Your STM32F303VCT6: **256 KB flash, 40 KB SRAM**. The book's STM32F091RC: 256 KB flash, 32 KB SRAM. Both integrated, no external memory.

Note what that ratio means. You have roughly six times more program space than data space. Running out of flash is unusual in a student project; running out of RAM is common, and a single carelessly sized `float` buffer can do it. Your lab manual makes this point in Lab 10 and it is worth taking seriously.

3.2 The program's memory layout

Given this C:

```

1 | int a, b;                /* uninitialised static */
2 | const char c = 123;      /* constant          */
3 | int d = 31;              /* initialised static */
4 |
5 | void main(void) {
6 |     int e;                /* automatic          */
7 |     char f[32];           /* automatic array     */
8 |     e = d + 7;
9 |     a = e + 29999;
10 |    strcpy(f, "Hello!");
11 | }
```

Things sort into two chips by two questions: *does it change?* and *does it have a fixed address?*

Region	Lives in	Holds
.text	Flash	Program instructions
Constant data	Flash	const values, string literals like "Hello!"
Initialisation data	Flash	The <i>initial values</i> of initialised statics, e.g. the 31 for d
Startup / library code	Flash	Reset_Handler, runtime library
Static data	SRAM	a, b, d — fixed addresses, whole program lifetime
Heap	SRAM	malloc'd blocks, grows one way
Stack	SRAM	Activation records: e, f, return addresses. Grows the other way

How an initialised variable gets initialised

`int d = 31;` needs the value 31 in RAM at d's address, but RAM has no contents at power-up. So the 31 is stored **in flash**, and startup code **copies it into RAM** before main runs.

So an initialised global costs you *both* flash and RAM. An uninitialised one costs only RAM.

Uninitialised statics are zeroed at startup, because their addresses are fixed and known so the startup code can clear the whole region. **Uninitialised automatics are not**, because their stack memory was probably used by something else moments ago. That is the mechanism behind "reading an uninitialised local gives garbage", and the garbage is specifically the remains of a previous function's locals.

3.3 Start-up code

Recall from Week 07 that the CPU loads SP from address 0 and PC from address 4, so the first code to run is `Reset_Handler` in the startup assembly file. It has real work to do before main is viable: configure and select a high-speed oscillator once it has stabilised, set the vector table location, reset the PLL configuration, then copy the initialisation data from flash to RAM, zero the uninitialised statics, and finally call `__main`.

Listing 1: From the startup file. Note `SystemInit` runs before `__main`.

```

1 Stack_Size EQU 0x00000400 ; 1 KB of stack
2 Stack_Mem AREA STACK, NOINIT, READWRITE, ALIGN=3
3 __initial_sp SPACE Stack_Size
4 ; label = top of stack
5
6 Reset_Handler PROC
7 EXPORT Reset_Handler [WEAK]
8 IMPORT __main
9 IMPORT SystemInit
10 LDR R0, =SystemInit
11 BLX R0 ; clocks, PLL, vector table
12 LDR R0, =__main
13 BX R0 ; never returns
14 ENDP

```

Beware the trap

See `Stack_Size EQU 0x00000400`? That is your entire stack, 1 KB, set at build time by a constant in an assembly file. Nothing checks it at runtime on a Cortex-M0.

So a deep call chain, or one function with a large local array, silently walks the stack pointer down past the bottom of the stack region and into whatever is below, which is usually your global variables. The symptom is a global changing value for no reason, which is a genuinely horrible bug to chase.

This is the single most useful reason to know your program's memory layout: **a large local array is dangerous in a way that a large global is not.** `int buffer[2048];` as a global is a clear 8 KB you can see in the size report. The same line inside a function is 8 KB of stack that may not exist.

Note also that `main` in an embedded program **never returns**. There is no operating system to return to. That is why every example in this course ends in `while(1)`.

4 Deep dive 3: The AAPCS, or who owns which register

This is the section that makes the rest make sense.

When function A calls function B, both want to use the sixteen registers. Somebody has to give. The **Arm Architecture Procedure Call Standard** settles it by dividing the registers into two groups.

Register	Must be restored?	Special use	Group
R0	No	Argument 1, result 1	Scratch
R1	No	Argument 2, result 2	Scratch
R2	No	Argument 3	Scratch
R3	No	Argument 4	Scratch
R4–R7	Yes	—	Preserved
R8–R11	Yes	—	Preserved
R12	No	—	Scratch
SP (R13)	Yes	Top of stack	Preserved
LR (R14)	No	Return address after BL/BLX	Scratch
PC (R15)	No	Program counter	—

Read it as a contract with two clauses. **A caller must assume R0–R3, R12, and LR are destroyed** by any call, so anything valuable in them gets saved first. **A callee must leave R4–R11 and SP exactly as it found them**, so if it wants to use one, it saves and restores it itself.

Arguments go in R0–R3, in order, because registers are far faster than memory. A fifth argument onward goes on the stack. Return values up to 32 bits come back in R0; up to 64 bits in R0 and R1; anything larger on the stack.

Why this explains Week 07

Recall that exception entry pushes exactly `xPSR, PC, LR, R12, R3, R2, R1, R0`.

Compare that list against the table. It is **precisely the scratch group**, plus the flags and the return address. The hardware saves exactly what a C function is permitted to destroy, which is why an ISR can be an ordinary C function: the compiler's normal prologue

already handles R4–R11, and the hardware handled the rest.

The two facts are the same fact. Interrupt entry was designed around the calling convention.

5 Deep dive 4: Prolog, body, epilog

Every compiled function has three parts.

The three sections, and the activation record

Prolog: saves preserved registers and prepares the activation record.

Body: does the work the source described.

Epilog: restores preserved registers, places the return value, frees the activation record, returns.

Activation record (or stack frame): temporary memory for one function's preserved registers, arguments, locals, and return address. It exists *only* from the function's start to its end.

The **call stack** is the stack of activation records for functions that have started but not finished. While `main` calls `J` which calls `B`, the call stack holds three frames.

Take this function:

```
1 int fun4(char a, int b, char c) {
2     int x[8];
3     x[0] = a * b;
4     x[c] = b;
5     return a + b + c;
6 }
```

5.1 The prolog

Listing 2: Book Listing 5.4. Two instructions, two jobs.

```
1 fun4 PROC
2 0000ba b510      PUSH    {r4, lr}
3 0000bc b088      SUB     sp, sp, #0x20
```

`PUSH {r4, lr}` saves R4 because the body will use it and R4 is preserved, and saves LR because the body might call a subroutine, which would overwrite LR with a second return address and lose the first.

`SUB sp, sp, #0x20` allocates 32 bytes, which is exactly `int x[8]`: eight elements times four bytes. Recall from Week 06 that the stack grows toward smaller addresses, so *subtracting* from SP is how you allocate.

5.2 The epilog

Listing 3: Book Listing 5.5. Return value, deallocate, restore, return.

```
1 ;;;105      return a+b+c;
2 0000ca 1840      ADDS    r0, r0, r1      ; a + b, result heading for R0
3 0000cc 1880      ADDS    r0, r0, r2      ; + c
4 ;;;106      }
5 0000ce b008      ADD     sp, sp, #0x20    ; free the 32 bytes
6 0000d0 bd10      POP     {r4, pc}        ; restore R4 AND return
```

Four things worth noticing.

The two ADDS put the result in R0, per the convention. Note the arguments were already sitting in R0, R1, R2, so $a+b+c$ needs no loads at all.

ADD sp, sp, #0x20 deallocates by adding back exactly what the prolog subtracted. Allocation and deallocation must match, and the compiler guarantees they do.

And POP {r4, pc} is the clever one. It does two jobs in one instruction: restores R4 as the convention requires, and loads the *saved LR value into the PC*, which is a return. So the prolog's PUSH {r4, lr} and the epilog's POP {r4, pc} are a matched pair, and the asymmetry in the register names is deliberate rather than a typo.

Two ways to return

If a function calls nothing, its return address stays safely in LR for its whole life, so it returns with BX LR and needs no stack frame at all. These are **leaf functions** and they are cheap.

If a function calls anything, LR will be overwritten by that call, so the prolog must PUSH {LR} and the epilog returns with POP {PC}.

So you can tell at a glance from a disassembly whether a function makes calls. That is a genuinely useful debugging skill.

6 Deep dive 5: Exception handlers, compiled

Recall from Week 08 that an ISR is an ordinary C function. Three differences, though.

First, **no arguments and no return value**, enforced. The compiler marks a handler with the __irq qualifier and errors out if you give it either. CMSIS's handler declarations already carry __irq.

Second, because a handler can execute anywhere, **all registers must be treated as preserved**. The hardware covers R0–R3 and R12; the handler's prolog covers the rest.

Third, the return. Recall there is no return-from-interrupt instruction on ARMv6-M, so BX LR with the EXC_RETURN code in LR triggers the unwind. If the handler called a subroutine, LR was overwritten and the code was pushed, so you pop it into the PC instead.

Listing 4: Book Listing 5.6. A timer ISR's prolog and epilog.

```

1 TIM6_DAC_IRQHandler PROC
2 000000 b430 PUSH {r4, r5} ; body uses R0-R5; R4,R5 preserved
3 ; ... handler body ...
4 000086 bc30 POP {r4, r5}
5 000088 4770 BX lr ; EXC_RETURN triggers the unwind
6 ENDP

```

Compare against fun4. Same shape, but no LR in the PUSH, because this handler calls nothing, and BX lr rather than POP {pc}. The BX lr restores R0–R3, R12, LR, PC, and xPSR as a side effect of the hardware unwind.

Beware the trap

Notice what this means for ISR cost. A handler that uses many registers has a longer prolog and epilog, on top of the fixed 16-cycle hardware entry. A handler that calls a function also pushes LR, and that function has its own frame.

So "keep ISRs short" from Week 08 is not only about the instructions you can see. Calling a large library function from inside a handler, or touching floating point on your M4 so that the FPU registers get stacked too, adds cost at every single entry. That is a real consideration for your Lab 07 filtering work.

7 Deep dive 6: Control flow, with the actual assembly

Here is the part the book's exercises are made of. In every listing below, R1 holds x and R2 holds y .

7.1 if / else

Listing 5: if (x) y++; else y-;

```

1  ;;;39      if (x){
2  000056    2900      CMP      r1, #0          ; sets flags; Z=1 if x == 0
3  000058    d001      BEQ      |L1.94|         ; if equal, jump to else
4  ;;;40      y++;
5  00005a    1c52      ADDS     r2, r2, #1
6  00005c    e000      B        |L1.96|         ; skip the else
7  |L1.94|
8  ;;;42      y--;
9  00005e    1e52      SUBS     r2, r2, #1
10 |L1.96|          ; merge point

```

The pattern is: **compare, branch on the opposite condition to the else, true-case code, unconditional branch to the merge point, else-case code, merge point.**

Note that BEQ branches when $x == 0$, which is the *false* case in C. Compilers invert the test so the true case falls straight through, which is fractionally faster.

7.2 switch

Listing 6: Cases 1, 31, and default. Chain of comparisons.

```

1  000060    2901      CMP      r1, #1
2  000062    d002      BEQ      |L1.106|         ; case 1
3  000064    291f      CMP      r1, #0x1f        ; 31 in hex
4  000066    d104      BNE      |L1.114|         ; neither -> default
5  000068    e001      B        |L1.110|         ; case 31
6  ...
7  000072    1e52      SUBS     r2, r2, #1        ; default: y--
8  000074    bf00      NOP                      ; debug placeholder
9  |L1.118|          ; merge point

```

Three ways to compile a switch

Comparison chain, as above. Simple, and the cost depends on which case matches: the last case is the slowest.

Jump table: an array of addresses indexed by the switch value. **Constant time** regardless of which case matches, but it costs memory and only works when the case values are dense.

Computed jump: calculate the target address arithmetically.

The compiler picks based on how many cases there are and how spread out their values are. Which is why **you cannot assume a switch is $O(1)$** , and why an FSM's switch in a tight control loop is worth checking in the disassembly if timing is critical.

7.3 do / while: bottom test

Listing 7: do { x += 2; } while (x < 20); The simplest loop.

```

1  000078    bf00      NOP
2  |L1.122|

```



```

3 | 00007a 1c89      ADDS    r1, r1, #2      ; body
4 | 00007c 2914      CMP     r1, #0x14      ; test against 20
5 | 00007e d3fc      BCC     |L1.122|        ; branch if carry clear = if less

```

Body first, then test. **One branch per iteration**, which makes this the cheapest loop shape.

BCC means "branch if carry clear". CMP computes a subtraction without storing the result, and for unsigned values a clear carry means a borrow happened, meaning the first operand was smaller. So carry clear = less than.

7.4 while: top test, laid out backwards

Listing 8: while (x<10) { x = x + 1; } Note the jump on entry.

```

1 | 000080 e000      B        |L1.132|        ; jump straight to the TEST
2 |          |L1.130|
3 | 000082 1c49      ADDS    r1, r1, #1      ; body
4 |          |L1.132|
5 | 000084 290a      CMP     r1, #0xa        ; test
6 | 000086 d3fc      BCC     |L1.130|        ; back to the body if x < 10

```

Why the layout is inverted

A while loop must test *before* the first iteration. The naive layout puts the test at the top and needs **two** branches per iteration: one for the test, one to jump back.

So the compiler puts the **body first and the test at the bottom**, then adds a single B on entry to skip over the body to the test. Now steady-state iterations cost **one** branch, exactly like do/while, and you pay one extra branch once, on entry.

So a while loop's assembly is laid out in the opposite order to its source. This trips people up reading disassembly for the first time, and it is a very likely exam question: *why does the while loop begin with an unconditional branch?*

7.5 for: four parts

Listing 9: for (i = 0; i < 10; i++) { x += i; } R3 holds i.

```

1 | 000088 2300      MOVVS   r3, #0          ; init: i = 0
2 | 00008a e001      B        |L1.144|        ; jump to test
3 |          |L1.140|
4 | 00008c 18c9      ADDS    r1, r1, r3      ; body: x += i
5 | 00008e 1c5b      ADDS    r3, r3, #1      ; update: i++
6 |          |L1.144|
7 | 000090 2b0a      CMP     r3, #0xa        ; test: i < 10
8 | 000092 d3fb      BCC     |L1.140|

```

Four parts: init, test, body, update. Laid out as **init, body, update, test**, executed as **init, test, body, update, test, ...** Same trick as while, one more moving piece.

7.6 Calling a subroutine

Listing 10: n = fun5(1,2,3,4); Arguments into R0-R3.

```

1 | MOVS    r0, #1      ; argument 1
2 | MOVS    r1, #2      ; argument 2
3 | MOVS    r2, #3      ; argument 3
4 | MOVS    r3, #4      ; argument 4
5 | BL      fun5        ; branch and link: LR = return address
6 | STR     r0, [sp, #0] ; result came back in R0, store to local n

```

Exactly the AAPCS table, made concrete.

8 Deep dive 7: Accessing data, four ways

A variable must be in a register before the CPU can operate on it. How it gets there depends on where it lives.

8.1 Static: PC-relative, because addresses do not fit

Here is a real constraint. A static variable can be anywhere in the 32-bit address space, so its address needs 32 bits. But most Thumb instructions are only **16 bits** long. There is no room for a 32-bit address *and* an opcode.

The solution is **PC-relative addressing**. The compiler stores the variable's 32-bit address in a constant pool in flash, physically near the instructions that use it, then loads it into a register with an LDR whose offset from the PC is small enough to fit.

Listing 11: siA = 2; Three instructions for one assignment.

```

1 | 00000e 2102      MOVS    r1, #2          ; the value
2 | 000010 4a49      LDR     r2, |L1.312|      ; load siA's ADDRESS from the pool
3 | 000012 6011      STR     r1, [r2, #0]      ; store the value at that address
4 | ...
5 | |L1.312| DCD     siA                    ; the constant pool entry

```

Two loads happen conceptually: one for the address, one for the data. That is why a static variable access is more expensive than a local.

8.2 Automatic: SP-relative

Locals live in the activation record, at a fixed offset from SP, written [sp, #offset]. The offset must be a multiple of four and lies between 0 and 1020.

Listing 12: int aiE=4, aiF=5, aiG=6; One store each, no address load.

```

1 | 000002 2104      MOVS    r1, #4
2 | 000004 9102      STR     r1, [sp, #8]      ; aiE
3 | 000006 2105      MOVS    r1, #5
4 | 000008 9101      STR     r1, [sp, #4]      ; aiF
5 | 00000a 2106      MOVS    r1, #6
6 | 00000c 9100      STR     r1, [sp, #0]      ; aiG

```

Cheaper than static: no address to load, because SP already holds the base. **This is a real argument for preferring locals**, on top of the modularity argument.

8.3 Pointers

Listing 13: apD = &aiB; then (*apD)++;

```

1 | ;;;22          apD = &aiB;
2 | 00001e a803      ADD     r0, sp, #0xc      ; address of aiB = SP + 0x0C
3 | ;;;23          (*apD)++;
4 | 000020 6801      LDR     r1, [r0, #0]      ; load through the pointer
5 | 000022 1c49      ADDS    r1, r1, #1       ; increment
6 | 000024 6001      STR     r1, [r0, #0]      ; store back through it

```

Beware the trap

Look at those last three instructions. LDR, ADDS, STR.

That is the read-modify-write from Week 08, in the flesh. (*apD)++ is one C expression and three memory operations, and an interrupt landing between them produces exactly the lost-increment bug from the Week 08 table. Seeing it as assembly is the point: the race is not an abstraction, it is these three lines.

8.4 Arrays, and the address arithmetic

An element's address is the array's base plus an offset. For a 1D array the offset is the index times the element size.

Listing 14: `i = buff2[0] + buff2[n]`; `buff2` is unsigned char, so size 1.

```

1 | 00009e 4602      MOV     r2, r0          ; n arrived in R0, keep it in R2
2 | 0000a0 4827      LDR     r0, |L1.320|    ; base address of buff2
3 | 0000a2 7800      LDRB    r0, [r0, #0]    ; buff2[0], constant offset
4 | 0000a4 4b26      LDR     r3, |L1.320|    ; base again
5 | 0000a6 5c9b      LDRB    r3, [r3, r2]    ; buff2[n], register offset
6 | 0000a8 18c0      ADDS    r0, r0, r3
7 | 0000aa 9000      STR     r0, [sp, #0]    ; store to local i

```

Note LDRB for a byte, and note that `buff2[0]` uses a constant offset the compiler folded, while `buff2[n]` needs a register offset.

Now the 2D case, which is the arithmetic worth understanding. Declared short `int buff3[5][7]`: five rows, seven columns, two bytes per element, $2 \times 5 \times 7 = 70$ bytes total, laid out **row by row**.

$$\text{address} = \text{base} + \underbrace{n \times (7 \times 2)}_{\text{row offset}} + \underbrace{j \times 2}_{\text{column offset}}$$

Listing 15: `i += buff3[n][j]`; Six instructions of address arithmetic.

```

1 | 0000ac 200e      MOVVS   r0, #0xe       ; row size = 7 * 2 = 14 bytes
2 | 0000ae 4350      MULS    r0, r2, r0      ; row offset = n * 14
3 | 0000b0 4b24      LDR     r3, |L1.324|    ; base address of buff3
4 | 0000b2 18c0      ADDS    r0, r0, r3      ; base + row offset
5 | 0000b4 004b      LSLS    r3, r1, #1         ; column offset = j * 2, via shift
6 | 0000b6 5ac0      LDRH    r0, [r0, r3]    ; load halfword at base+row+col
7 | 0000b8 9b00      LDR     r3, [sp, #0]    ; load i
8 | 0000ba 18c0      ADDS    r0, r0, r3
9 | 0000bc 9000      STR     r0, [sp, #0]    ; store i

```

Two things to take from this listing

Multiply by a power of two becomes a shift. `LSLS r3, r1, #1` is $j \times 2$, and a shift is one cycle while `MULS` is more. The row offset needed `MULS` because 14 is not a power of two.

A 2D array access costs about six instructions of pure arithmetic before the data load even happens. In a tight loop, hoisting the row-base calculation out of the inner loop is a real optimisation, and this listing is why. Recall Lab 02's matrix multiplication task: that nested loop is doing this arithmetic on every iteration.

In the lab

This is exactly the ground Lab 10 stands on. It asks you to investigate the stack and the heap, then write your own allocator using direct SRAM addressing.

The connections: your heap driver's fixed heap region is *static* memory, addressed the PC-relative way. The block bitmap you index is *array* access, with the address arithmetic above. Your `heap_alloc` returning a pointer is a *pointer* in the sense of Section 7.3, and the reason `malloc` is discouraged in embedded code is partly what this chapter shows: a heap block's address is not known at compile time, so every access through it costs the extra indirection, and none of it can be checked by the linker.

The manual's advice to prefer static buffers for fixed-size needs is the same advice, arrived

at from the memory-layout side.

And that is a wrap for the compiler. Back to hardware next week.

9 Tying it back

We asked what the compiler actually produces, and the answer has a consistent logic once you see the constraint behind each piece.

Five tools translate and place your code, and the compiler is free to produce any object code that is behaviourally correct, which is why you cannot time your program by counting C statements and why `volatile` has to exist. Your variables sort into flash and RAM by whether they change and whether they have a fixed address, with initialised globals costing both because their initial values are copied out of flash at startup, and uninitialised locals holding garbage because their stack space was somebody else's a moment ago. The AAPCS divides the registers into scratch and preserved, which determines what a prolog must push, and which turns out to be exactly the list the interrupt hardware stacks for you. Every function is a prolog that pushes and allocates, a body, and an epilog that computes the return value into `R0`, deallocates, and returns, often with a single `POP {r4, pc}` doing restore-and-return together. Control flow compiles to compare-and-branch, with loops laid out body-first and an entry jump to the test, so that steady-state iterations cost only one branch. And data access costs depend on where the variable lives: a local is one SP-relative instruction, a static needs its 32-bit address fetched from a constant pool because a 16-bit Thumb instruction cannot hold one, and a 2D array element costs six instructions of arithmetic before you even touch the data.

Next week we go back to hardware and to the physical world, with the ADC: how a continuous voltage becomes a number, what quantisation and sampling cost you, and why your F303's ADC is a substantially better instrument than the book's.

Cheat sheet: Week 10

Toolchain

Compiler `.c` → `.s` · **Assembler** `.s` → `.o` · **Linker** `.o` + `libs` → `.axf` · **Programmer** writes flash · **Debugger** inspects.

Compiler steps: preprocess, check syntax, intermediate code, high-level optimise, generate assembly + allocate registers, low-level optimise, emit `.s`.

Compilation is not one-to-one. Many correct compilations exist. You cannot time code by counting C statements.

Assembly file elements: **instruction**, **comment** `(;)`, **symbol** (a name for a value), **label** (a name for an address).

Memory

Flash: `.text`, `const` data, string literals, **initial values** of initialised statics, startup code.

SRAM: static data, heap (grows up), stack (grows down).

An initialised global costs flash *and* RAM; the value is copied out of flash at startup.

Uninitialised statics are zeroed (fixed addresses). **Uninitialised automatics are garbage** (reused stack).

EEPROM erases one location at a time; **flash erases a whole page**. SRAM is latches; DRAM is capacitors, needs refresh, smaller and cheaper.

Stack size is a **build-time constant** (`Stack_Size EQU 0x400`) with no runtime check. A large local array can silently overrun into your globals. A large *global* is visible in the size report; a large *local* is not.

`main` never returns in an embedded program.

AAPCS, and this table explains half the chapter

Scratch (caller must assume destroyed): R0–R3, R12, LR.

Preserved (callee must save and restore): R4–R11, SP.

Arguments in R0–R3 in order, then the stack. Return: R0 up to 32 bits, R0+R1 up to 64, stack beyond.

Exception entry stacks exactly the scratch set plus xPSR and PC. That is why an ISR is an ordinary C function.

Function structure

Prolog: `PUSH {r4, lr}` saves preserved registers and LR; `SUB sp, sp, #n` allocates locals.

Epilog: result into R0; `ADD sp, sp, #n` deallocates; `POP {r4, pc}` restores *and* returns.

Activation record / stack frame = one function's preserved registers, args, locals, return address. Exists only for that function's lifetime. **Call stack** = all live frames.

Leaf function (calls nothing): keeps the return address in LR, returns with `BX LR`, may need no frame.

Calls something: must `PUSH {LR}`, returns with `POP {PC}`. *You can spot this in a disassembly.*

ISRs: `__irq` qualifier, no args, no return value, all registers effectively preserved, returns via `BX LR` with `EXC_RETURN`.

Control flow

`if/else:` `CMP`, branch on the *inverted* condition to the else, true code, `B` to merge, else code,

merge.

switch: comparison chain (cost depends on which case), **jump table** (constant time, costs memory, needs dense values), or computed jump. **Not guaranteed $O(1)$.**

do/while: body then test. **One branch per iteration**, the cheapest shape.

while: laid out **body then test**, with an B on entry that jumps to the test. So steady state is also one branch. *This is why a while loop's assembly starts with an unconditional branch.*

for: laid out init, body, update, test. Executed init, test, body, update, test. . .

BCC = branch if carry clear = unsigned "less than", because CMP is a subtraction and a clear carry means a borrow.

Data access, cheapest to dearest

Automatic: [sp, #offset], offset a multiple of 4, range 0–1020. One instruction, SP is already the base.

Static: needs **PC-relative addressing**, because a 32-bit address does not fit in a 16-bit Thumb instruction. The address sits in a constant pool near the code; LDR fetches it, then a second access reaches the data.

Pointer: LDR, operate, STR. *(*p)++ is literally the Week 08 read-modify-write race.*

1D array: base + index $\times$ element size. LDRB/LDRH/LDR by size.

2D array a[R][C] of s -byte elements: $\text{addr} = \text{base} + n \times (C \times s) + j \times s$. About **six instructions of arithmetic** per access. Multiply by a power of two becomes LSLs.

Likely exam prompts from this chapter

- Given an assembly listing, explain each instruction and say what is in each register. *This is the entire Chapter 5 exercise set.*
- Which registers must a subroutine preserve, and which may it destroy?
- What does the prolog do, and what does the epilog do?
- Why does POP {r4, pc} both restore a register and return?
- Why does a while loop's assembly begin with an unconditional branch?
- Why is PC-relative addressing needed for static variables?
- Compute the address of buff[n][j] given the dimensions and element size.
- Why are uninitialised globals zero but uninitialised locals garbage?